Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №4
з дисципліни «Проектування програмних систем для мобільних пристроїв»

«Емулятор та ресурси Android»

Варіант №4

Виконав:
студент 2 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Нестерук А. О.

Дата здачі: 20.10.2026

КИЇВ – 2026

Мета. Вивчити програмні засоби емуляції операційної системи Android, розглянути її ресурси, основні методи роботи із файловою системою, ознайомитись з методами підключення бібліотек та віджетів у додаток, вивчити маніфест операційної системи.

Завдання. Додати елементи управління (`Button`, `TextView`, `EditText`, `ListView`) в програму, розроблену в лабораторній роботі №3. Запустити програму на емуляторі.

Завдання за варіантом. Доменна область: Абітурієнт.

Поля: ID, Прізвище, Ім’я, По-батькові, Адреса, Телефон, Оцінки.

Створити масив об’єктів.

Вивести:

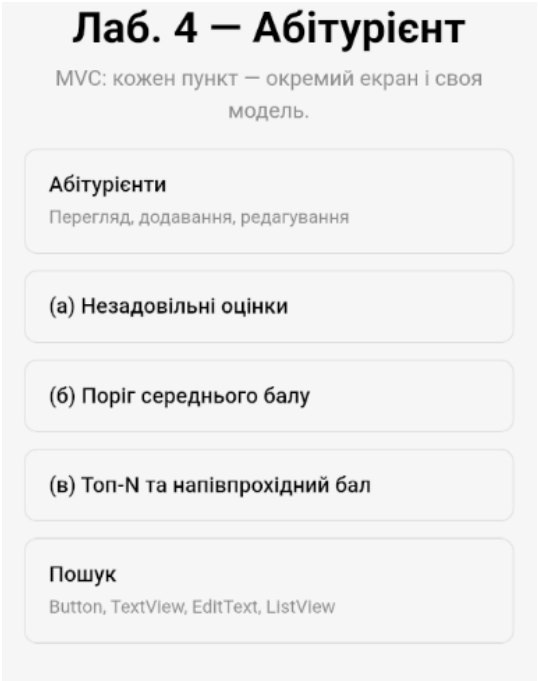
а. список абітурієнтів, які мають незадовільні оцінки;

б. список абітурієнтів, середній бал у яких вище заданого;

в. вибрати задане число `N` абітурієнтів, що мають найбільш високій середній бал (вивести також повний список абітурієнтів, які мають напівпрохідний бал).

Хід роботи.

Для виконання роботи використано застосунок, розроблений у третій лабораторній роботі, без зміни архітектури MVC чи стеку – Rust, Tauri, React з TypeScript. Для даної роботи створено новий окремий екран – «Пошук», на якому всі чотири елементи (`Button`, `TextView`, `EditText` та `ListView`) використано явно.



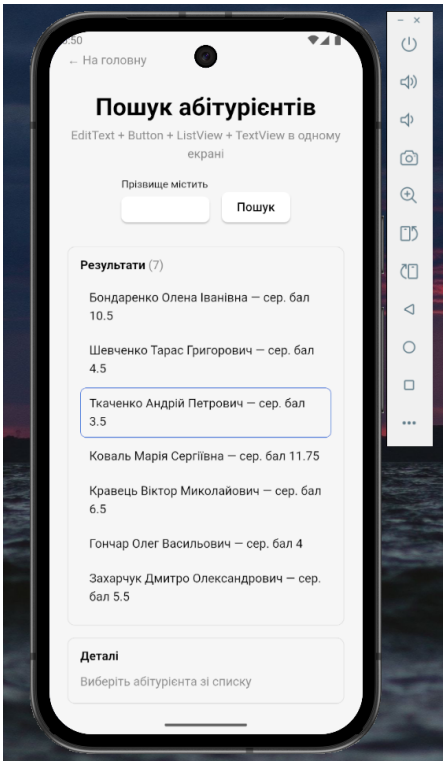
На боці Rust додано контролер-модуль `search` із відповідною командою, яка фільтрує список абітурієнтів репозиторію за входженням підрядка в прізвище без врахування регістру і повертає результат у тому ж вигляді, що й список абітурієнтів на екрані «Абітурієнти». На боці React створено відповідний «екран», що поєднує всі чотири елементи.

`EditText` – текстове поле, контрольоване React-станом через атрибути `value` та `onChange`. Це функціональний аналог `TextWatcher`/`addTextChangedListener` в Android – кожна зміна тексту одразу оновлює стан JS, так само як `TextWatcher` сповіщає Activity про зміну вмісту поля.

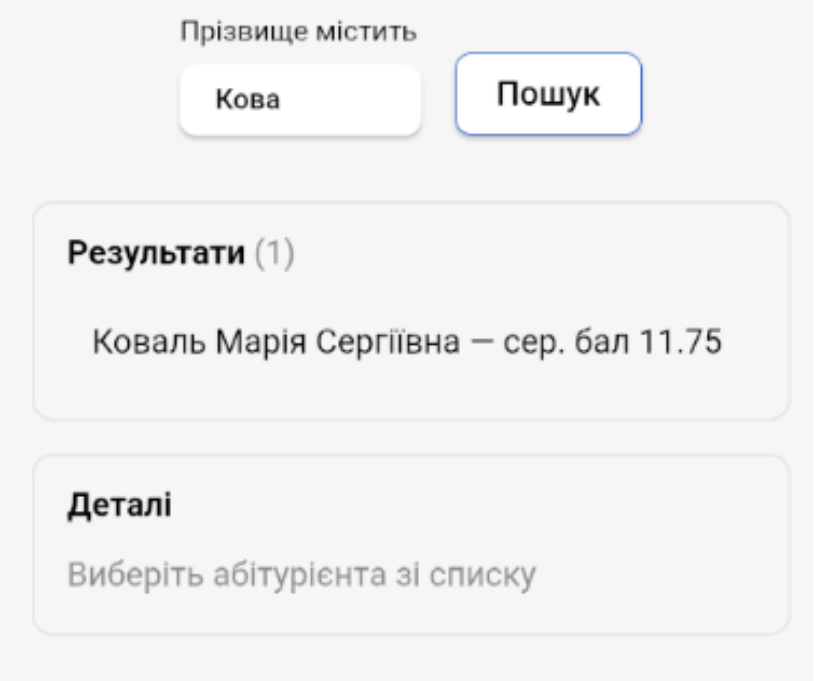
`Button` – кнопка «Пошук», що за натисканням викликає відповідну функцію. Це аналог `setOnClickListener` на `android.widget.Button`.

`ListView` – результати виведено не таблицею, як на інших екранах, а списком клікабельних елементів, по одному на абітурієнта. Клік на елементі зберігає його `ID` у стані `selectedId`. Це ближчий аналог `android.widget.ListView` з `Adapter`'ом та `setOnItemClickListener`, ніж таблиця – один елемент інтерфейсу на один рядок даних, з власним обробником кліку.

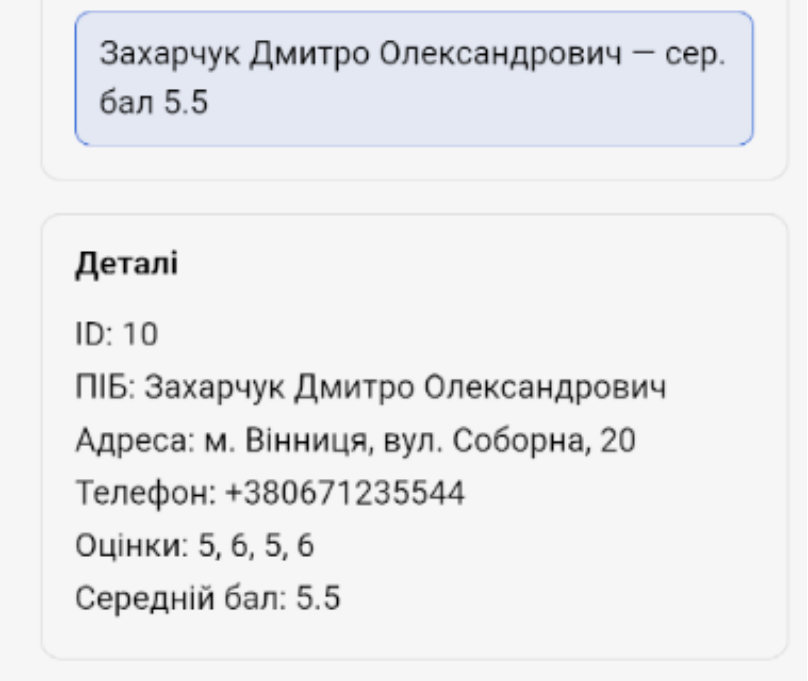
`TextView` – під списком розміщено окрему секцію «Деталі», яка виводить повну інформацію про вибраного абітурієнта (ID, ПІБ, адреса, телефон, оцінки, середній бал) лише для читання. Блок оновлюється декларативно при зміні `selectedId`, що є React-аналогом імперативного виклику `textView.setText(...)` всередині обробника кліку на елементі списку в Android.



Коректність реалізації перевірено на емуляторі Android – уведено частину прізвища в поле пошуку, натиснуто кнопку «Пошук», і список скоротився до абітурієнтів, чиє прізвище містить введений підрядок.



Після натискання на один з елементів списку під ним з’являється детальна інформація про вибраного абітурієнта.



Висновок. У цій лабораторній роботі до застосунку «Абітурієнт» додано елементи керування `Button`, `TextView`, `EditText` та `ListView`, явно виокремлені в новий екран «Пошук». Наведено відповідність кожного React-елемента нативному Android-віджету. Коректність роботи підтверджено на емуляторі Android.

Контрольні запитання.

1. **Дайте визначення поняття «емулятор».** Це апаратна або програмна система, яка точно моделює поведінку іншої системи, дозволяючи виконувати програми або використовувати периферійні пристрої, призначені для оригінальної архітектури, на цільовій платформі. Головна мета емуляції полягає у відтворенні внутрішньої структури та логіки роботи пристрою на низькому рівні, що відрізняє її від симуляції, яка лише імітує зовнішню абстрактну поведінку.
2. **Назвіть основні етапи встановлення емулятора Android.** Процес встановлення зазвичай відбувається через інтегроване середовище Android Studio, де спочатку необхідно відкрити SDK Manager для завантаження потрібних версій Android SDK та системних образів, а потім перейти до Device Manager для створення нового віртуального пристрою, обравши апаратний профіль, версію операційної системи та налаштувавши параметри продуктивності, такі як обсяг оперативної пам'яті та графічне прискорення.
3. **Наведіть види емуляторів. Назвіть їх переваги та недоліки.** Емулятори поділяються на апаратні та програмні, а також за рівнем системи на емулятори архітектури процесора та емулятори повноцінної операційної системи. Їхня головна перевага полягає у можливості тестувати програмне забезпечення на різноманітних конфігураціях обладнання без необхідності купувати реальні фізичні пристрої, що суттєво економить кошти та спрощує процес зневадження, проте недоліком є висока вимогливість до ресурсів хостової машини, можливе зниження швидкодії через трансляцію інструкцій та іноді неповна або неточна підтримка специфічних апаратних функцій, таких як датчики чи камери.
4. **Назвіть основні ресурси Android.** Основні ресурси в Android-додатках зберігаються у каталозі `res` і включають макети користувацького інтерфейсу у форматі XML, строкові змінні для інтернаціоналізації, графічні зображення різної роздільної здатності, кольори, стилі та теми, а також анімації і сирі бінарні файли, що в сукупності дозволяє суворо відокремити логіку програмного коду від представлення та полегшує адаптацію програми під різні конфігурації екранів пристроїв.
5. **Назвіть методи роботи з файлами.** Робота з файлами в Android базується на стандартних класах мови Java або Kotlin з використанням потоків вводу-виводу, при цьому операційна система надає специфічні методи контекс-

ту, наприклад `openFileInput` та `openFileOutput` для безпечного доступу до внутрішнього ізольованого сховища додатку, а також окремі інструменти та API для роботи із зовнішнім сховищем, кеш-пам'яттю та файлами з каталогу ресурсів або `assets`.

6. **Охарактеризуйте формат зберігання даних JSON.** Це легкий текстовий формат обміну даними, який легко читається людьми та ефективно парситься машинами, що базується на двох основних структурах: колекціях пар ключ-значення, які представляють об'єкти, та впорядкованих структурах значень, які представляють масиви, що робить його універсальним стандартом для передачі даних у веб-сервісах та клієнт-серверній архітектурі незалежно від мови програмування.
7. **Назвіть основні етапи підключення бібліотек до Android Studio.** Підключення зовнішніх бібліотек найчастіше здійснюється через систему збирання Gradle, що вимагає відкриття файлу `build.gradle` на рівні модуля, додавання необхідної залежності з вказанням її координат у блок `dependencies` та подальшої синхронізації проекту для завантаження артефактів, хоча також можливе ручне додавання локальних компільованих файлів безпосередньо в директорію `libs` із відповідним оновленням конфігурації збирання.
8. **Назвіть призначення файлу `manifest.xml` та охарактеризуйте його.** Файл `AndroidManifest.xml` є ключовим конфігураційним документом будь-якого Android-додатку, який надає операційній системі життєво важливу метаінформацію перед запуском коду, визначаючи простір імен програми, її головні компоненти, такі як активності, фонові сервіси та приймачі широкомовних повідомлень, а також чітко декларуючи необхідні дозволи для доступу до захищених API, апаратних функцій та мінімальні вимоги до версії SDK.

Лістинг.

search.rs

```rust
//! Lab 4: a separate screen that explicitly
//! demonstrates `Button`, `TextView`, `EditText`, and `ListView`.

// State<T> must be taken by value in every command here — that's Tauri's
// calling convention, not a real ownership need.
#![allow(clippy::needless_pass_by_value, reason = "Tauri's convention")]

use crate::applicant::ApplicantView;
use crate::repository::Repository;
use std::sync::{Mutex, PoisonError};
use tauri::State;

#[tauri::command]
pub fn search_applicants(
state: State<'_, Mutex<Repository>>, query: String,
) -> Vec<ApplicantView> {
    let repo = state.lock().unwrap_or_else(PoisonError::into_inner);
    let needle = query.trim().to_lowercase();

    repo.list()
    .iter()
    .filter(|a| needle.is_empty() || a.last_name.to_lowercase().contains(&needle))
    .map(ApplicantView::from)
    .collect()
}
```

Search.tsx

```tsx
import { useEffect, useState } from "react";
import { invoke } from "@tauri-apps/api/core";
import { Link } from "react-router-dom";
import type { Applicant } from "../types";

// Demonstrates a Button, TextView, EditText, and ListView on a single screen (Lab 4 assignment).
// An EditText (search field) and a Button ("Search") filter the ListView (the list below);
// clicking an item in the list populates
// the TextView (a read-only details section) — all four elements are visible simultaneously.
export function Search() {
    const [query, setQuery] = useState("");
    const [rows, setRows] = useState<Applicant[] | null>(null);
    const [selectedId, setSelectedId] = useState<number | null>(null);

    async function search() {
        const result = await invoke<Applicant[]>("search_applicants", { query });
        setRows(result);
        if (!result.some((a) => a.id === selectedId)) {
            setSelectedId(null);
        }
    }

    useEffect(() => {
        search();
    }, []);

    const selected = rows?.find((a) => a.id === selectedId) ?? null;

    return (
    <main className="container">
```

```
        <Link className="back" to="/">
        ← На головну
        </Link>
        <h1>Пошук абітурієнтів</h1>
        <p className="subtitle">
        EditText + Button + ListView + TextView в одному екрані
        </p>

        {/* EditText + Button */}
        <form
        className="controls"
        onSubmit={(e) => {
                e.preventDefault();
                search();
        }}
        >
        <label>
        Прізвище містить
        <input
        type="text"
        value={query}
        onChange={(e) => setQuery(e.currentTarget.value)}
        />
        </label>
        <button type="submit">Пошук</button>
        </form>

        {/* ListView */}
        <section className="panel">
        <h2>
        Результати <span className="count">({rows?.length ?? 0})</span>
        </h2>
        {rows && rows.length > 0 ? (
            <ul className="list-view">
            {rows.map((a) => (
                <li
                key={a.id}
                className={a.id === selectedId ? "selected" : undefined}
                onClick={() => setSelectedId(a.id)}
                >
                {a.last_name} {a.first_name} {a.patronymic} — сер. бал{" "}
                {a.average}
                </li>
                ))}
            </ul>
            ) : (
            <p className="empty">— немає —</p>
            )}
        </section>

        {/* TextView */}
        <section className="panel">
        <h2>Деталі</h2>
        {selected ? (
            <p className="text-view">
            ID: {selected.id}
            <br />
            ПІБ: {selected.last_name} {selected.first_name}{" "}
            {selected.patronymic}
            <br />
            Адреса: {selected.address}
            <br />
            Телефон: {selected.phone}
            <br />
```

```
          Оцінки: {selected.grades.join(", ")}
          <br />
          Середній бал: {selected.average}
          </p>
          ) : (
          <p className="empty">Виберіть абітурієнта зі списку</p>
          )}
      </section>
      </main>
      );
}
```